

# BITÁCORA DE EJECUCIÓN Y CIERRE — SPRINT 2

**Foco del Incremento:** Seguridad y Organización — Autenticación JWT, Roles, Categorías y Características

**Stack Tecnológico:** Java 17 / Spring Boot 3.5 / Spring Security 6 / MariaDB / React 19 / Vite

## 1. Resumen del Incremento (Scope)

El Objetivo del Sprint 2 se centró en dotar a la plataforma de una infraestructura robusta de seguridad (Autenticación y Autorización) y en robustecer la granularidad del catálogo. Se implementaron los flujos de registro de usuarios con confirmación asíncrona por email, inicio de sesión basado en tokens estructurados (JWT) y persistencia de sesión en el cliente. Asimismo, se desplegó el sistema modular de categorías funcionales con filtros dinámicos y el sistema de asignación de características (*amenities*) para los alojamientos.

## 2. Arquitectura del Sistema e Integración

### 2.1. Backend (Spring Boot + Spring Security 6)

Se expandió la arquitectura base mediante el desacoplamiento de servicios y la incorporación del motor de seguridad reactiva por anotaciones:

Controller → Service (Interface + Impl) → Repository → Entity / DTO

Matriz de Componentes Introducidos:

| Módulo     | Entidad (Entity) | Objetos de Transferencia (DTO)              | Capa de Servicio | Capa de Control (Controller) |
|------------|------------------|---------------------------------------------|------------------|------------------------------|
| Auth       | User             | RegisterRequest, LoginRequest, AuthResponse | AuthService      | AuthController               |
| Categories | Category         | CategoryDTO                                 | CategoryService  | CategoryController           |
| Features   | Feature          | FeatureDTO                                  | FeatureService   | FeatureController            |
| Users      | —                | UserDTO, RoleRequest                        | UserService      | UserController               |
| Email      | —                | —                                           | EmailService     | N/A (Interno)                |

- Refactorización de Interfaces:** Se eliminó el uso del prefijo "I" en las interfaces de servicios ( `IAuthService` → `AuthService` ), unificando el criterio de nomenclatura con las directrices de diseño limpio modernas.
- Estrategia JWT:** Firma basada en algoritmo simétrico HMAC-SHA256 ( `HS256` ) con un tiempo de expiración fijo de 8 horas. El secreto de firma se inyecta dinámicamente mediante la clave de entorno `APP_JWT_SECRET` .
- Seguridad Declarativa:** Activación de `@EnableMethodSecurity` . El control de acceso a operaciones de mutación (escritura, actualización, borrado) se delegó a nivel de método mediante la directiva `@PreAuthorize("hasRole('ADMIN')")` .
- Módulo de Notificaciones:** Integración de un servicio de mensajería SMTP. En entorno de desarrollo se acopló con el sandbox de *Mailtrap* para interceptación asíncrona de correos.

## 2.2. Frontend (React + Vite)

Evolución de la topología de la Single Page Application (SPA) para soportar estados globales distribuidos y diálogos declarativos:

```
src/
├── components/
│   ├── Header/Header.jsx      (Integración de perfil, avatar y redirección por rol)
│   └── ConfirmDialog.jsx      (Componente modal agnóstico y reutilizable de confirmación)
├── hooks/
│   ├── useAuth.js             (Consumo simplificado del estado de sesión)
│   └── useConfirmCancel.js    (Abstracción de rollback de formularios en flujos admin)
├── pages/
│   ├── Home/Home.jsx          (Inyección de carrusel de categorías y triggers de filtrado)
│   ├── LoginPage.jsx           (Pantalla de autenticación con control de errores)
│   ├── RegisterPage.jsx        (Formulario de registro con feedback reactivo inline)
│   ├── ProductDetail/
│   │   └── Admin/
│   │       ├── Admin.jsx       (Layout por pestañas: Alojamientos, Categorías, Características,
Usuarios)
│   │       ├── AdminCategories.jsx (CRUD de categorías extraído)
│   │       ├── AdminFeatures.jsx  (CRUD de características nuevo)
│   │       └── AdminUsers.jsx     (Panel de gestión de privilegios y escalado de roles)
│   └── context/
│       └── AuthContext.jsx       (Contexto de sesión global con Lazy Initialization del
LocalStorage)
└── services/
    └── api.js                    (Módulo fetch con interceptor para inyección de cabeceras Bearer)
```

### 3. Trazabilidad de Historias de Usuario (User Stories)

| ID     | Historia de Usuario                       | Componente / Vista UI | Endpoint Backend                                 | Criterio de Aceptación / Estado                                                                                          |
|--------|-------------------------------------------|-----------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| US #12 | Categorizar productos de forma modular.   | AdminCategories.jsx   | POST/GET/PUT/DELETE /api/categories              | CRUD operativo. El borrado físico se bloquea (400 Bad Request) si la categoría contiene alojamientos activos.            |
| US #13 | Registro unificado de cuentas de usuario. | RegisterPage.jsx      | POST /api/auth/register                          | Validación de campos obligatorios en cliente/servidor. Feedback visual instantáneo ante correos duplicados.              |
| US #14 | Identificación de usuarios (Login).       | LoginPage.jsx         | POST /api/auth/login                             | Intercambio exitoso por JWT. Mutación del Header para desplegar Avatar con las iniciales del usuario.                    |
| US #15 | Cierre de sesión seguro (Logout).         | Header.jsx            | N/A (Lado Cliente)                               | Destrucción del token en localStorage, reseteo del contexto de React y redirección automática al Home.                   |
| US #16 | Identificar y gestionar administradores.  | AdminUsers.jsx        | GET /api/users, PUT /api/users/{id}/role         | Listado maestro de cuentas. Asignación de roles con validación preventiva para evitar el auto-quitado de permisos.       |
| US #17 | Administrar catálogo de características.  | AdminFeatures.jsx     | POST/GET/PUT/DELETE /api/features                | Alta y edición de elementos de equipamiento asociando nombres e íconos.                                                  |
| US #18 | Visualizar características en el detalle. | ProductDetail.jsx     | Propiedad lodging.features                       | Despliegue estructurado en el bloque "¿Qué ofrece este lugar?" en grilla.                                                |
| US #19 | Notificación automática post-registro.    | EmailService          | N/A (Evento asíncrono)                           | Disparo automático de correo formal de bienvenida parametrizado con los datos del usuario registrado.                    |
| US #20 | Barra de filtrado por categorías en Home. | Home.jsx              | GET /api/categories, GET /api/lodgings?category= | Renderizado de chips/tags reactivos. Al activarse, filtran la grilla de recomendaciones mediante parámetros de consulta. |
| US #21 | Adición simplificada de categorías.       | AdminCategories.jsx   | POST /api/categories                             | Apertura de formulario controlado embebido en modal con validaciones de unicidad de nombre.                              |

## 4. Catálogo de Endpoints de la API REST

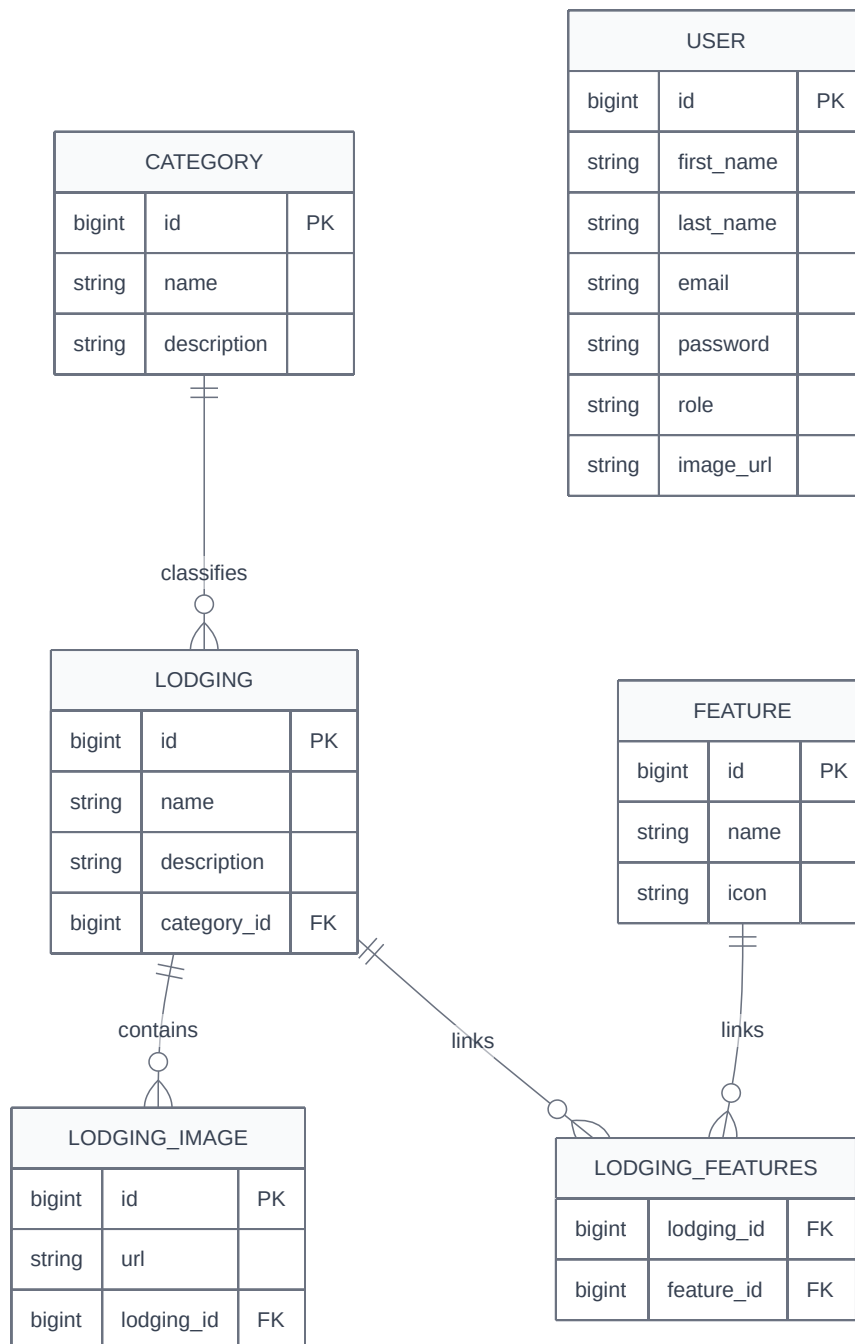
### 4.1. Endpoints de Negocio Heredados (Sprint 1)

| Método | Endpoint             | Acceso (RBAC) | Descripción                                           |
|--------|----------------------|---------------|-------------------------------------------------------|
| POST   | /api/lodgings        | ADMIN         | Crear alojamiento                                     |
| GET    | /api/lodgings        | Público       | Listar (soporta query params: ?page=&size=&category=) |
| GET    | /api/lodgings/random | Público       | Recomendaciones aleatorias                            |
| GET    | /api/lodgings/search | Público       | Búsqueda por nombre                                   |
| GET    | /api/lodgings/{id}   | Público       | Detalle del alojamiento                               |
| PUT    | /api/lodgings/{id}   | ADMIN         | Actualizar alojamiento                                |
| DELETE | /api/lodgings/{id}   | ADMIN         | Eliminar alojamiento                                  |

### 4.2. Endpoints de Seguridad y Organización (Sprint 2)

| Método | Endpoint             | Acceso (RBAC) | Descripción                                                   |
|--------|----------------------|---------------|---------------------------------------------------------------|
| POST   | /api/auth/register   | Público       | Registra usuario base con rol ROLE_USER. Retorna 201 Created. |
| POST   | /api/auth/login      | Público       | Autentica credenciales. Retorna 200 OK con JWT.               |
| POST   | /api/categories      | ADMIN         | Crea categoría (valida nombre único)                          |
| GET    | /api/categories      | Público       | Lista todas las categorías                                    |
| GET    | /api/categories/{id} | Público       | Categoría por ID                                              |
| PUT    | /api/categories/{id} | ADMIN         | Actualiza categoría                                           |
| DELETE | /api/categories/{id} | ADMIN         | Elimina categoría (bloqueado si tiene alojamientos)           |
| POST   | /api/features        | ADMIN         | Crea característica                                           |
| GET    | /api/features        | Público       | Lista características                                         |
| GET    | /api/features/{id}   | Público       | Característica por ID                                         |
| PUT    | /api/features/{id}   | ADMIN         | Actualiza característica                                      |
| DELETE | /api/features/{id}   | ADMIN         | Elimina característica                                        |
| GET    | /api/users           | ADMIN         | Lista usuarios registrados                                    |
| PUT    | /api/users/{id}/role | ADMIN         | Cambia rol (ROLE_USER ↔ ROLE_ADMIN)                           |

## 5. Modelo de Datos y Cardinalidad



- **Relación Lodging (M) ↔ (N) Feature** : Mapeada mediante tabla asociativa `lodging_features` . La eliminación de un alojamiento limpia sus registros asociativos.
- **Mapeo de Roles de Seguridad**: El campo `role` se persiste como `EnumType.STRING` . Spring Security mapea internamente estos valores bajo la convención `ROLE_USER` / `ROLE_ADMIN` , lo que permite el correcto funcionamiento de `@PreAuthorize("hasRole('ADMIN')")` .
- **Relación Lodging (N) → 1 Category** : FK `category_id` nullable. No se permite eliminar una categoría si tiene alojamientos vinculados.

## 6. Decisiones Técnicas Clave

- **Desacoplamiento de Servicios de Cuentas:** Se fragmentó la lógica de usuarios en dos componentes independientes: `AuthService` (procesa la autenticación, interactúa con el `AuthenticationManager` de Spring Security y despacha tokens) y `UserService` (abstrae operaciones CRUD de cuentas de usuario y reasignación de roles).
- **Estrategia de Persistencia de Pruebas Unitarias:** Uso sistemático de `@Transactional` en las clases de prueba de integración `@SpringBootTest`. Esto fuerza un *rollback* automático al término de cada test, previniendo que ejecuciones destructivas afecten datos persistentes en desarrollo.
- **Control Extendido de Cancelación de Flujos ( `useConfirmCancel` ):** En la capa de administración, se abstraio la lógica de interrupción en un Custom Hook de React. Si un operador realiza modificaciones en un formulario controlado y presiona "Cancelar", el hook evalúa si hubo cambios para desplegar un diálogo preventivo.
- **Gestión de Respuestas Vacías ( `204 No Content` ):** El cliente de servicios del frontend ( `api.js` ) se adaptó para procesar correctamente códigos de respuesta `204` en operaciones `DELETE`, evitando errores de parseo en respuestas sin cuerpo JSON.
- **Validación manual en formularios admin:** Se implementó el patrón `noValidate + validate() + fieldErrors` en todos los formularios del panel de administración, consistente con el enfoque utilizado en login y registro.
- **Containerización de la Base de Datos para Pruebas:** Se integró Testcontainers (v1.21.4) para levantar una instancia efímera de MariaDB 10.11 en Docker durante la ejecución de los tests de integración. La anotación `@ServiceConnection` de Spring Boot 3.5 inyecta automáticamente las credenciales del container, eliminando la dependencia de la base de datos de desarrollo y previniendo colisiones con registros preexistentes. Los tests unitarios (Mockito) no se ven afectados.

## 7. Limitaciones Conocidas y Deuda Técnica Controlada

1. **Persistencia Multimedia Estática:** La inyección de imágenes continúa supeditada a URLs de almacenamiento externo (picsum.photos). Sin carga real de archivos.
2. **Ciclo de Vida de Sesión Primitivo:** El sistema carece de un mecanismo de *Refresh Tokens*. Al cumplirse las 8 horas de validez del JWT, el token expira de forma directa, forzando al usuario a reautenticarse.
3. **Motores de Búsqueda y Puntuación en Espera:** La barra de texto del buscador público en el Home, el sistema de persistencia de alojamientos favoritos y los módulos de reseñas y puntuaciones siguen pendientes, agendados para su implementación en los Sprints 3 y 4.